CourtSy: technická dokumentace

Semestrální práce KTWA2 · Daniel Svoboda · říjen 2026 Repozitář `github.com/denysovski/ModernityRay` · Aplikace `denysovski.github.io/ModernityRay/`

CourtSy je frontendová aplikace fiktivního sportovního klubu: porovnání členství, rezervace kurtů, tréninkové programy, kalendář akcí, e-shop a objednávka. Backend neexistuje. Obsah je v datových modulech, stav v `localStorage`.

1. Framework a struktura

React 18 + Vite 5. React kvůli komponentám a hooku `useSyncExternalStore`, Vite kvůli rychlému vývoji a dělení kódu podle stránek. TypeScript není použitý (zadání ho jen doporučuje), typy popisuje **JSDoc**.

Složka	Obsah
`src/views/`	obrazovky, každá jako vlastní komponenta
`src/layouts/`	společný rám (navigace, košík, patička)
`src/components/`	sdílené prvky (pole formuláře, odkazy, obrázky, dialogy)
`src/lib/`	logika bez Reactu: stav, validace, SEO, formátování
`src/data/`	obsah webu (ceny, areály, programy, akce, zboží)
`src/routes.js`	všechny URL adresy na jednom místě

2. Routování a komponenty

React Router 6 (data router): 21 adres, obrazovky se načítají až při otevření (`lazy`), neznámá adresa zobrazí stránku 404. Vzor přehled → detail se opakuje u členství, areálů, programů a akcí.

Komponenty jsou ve čtyřech vrstvách:

1. **`main.jsx`** spustí aplikaci, obalí ji stavem a routerem.
2. **`RootLayout`** drží navigaci, košík a patičku. Při navigaci se nemění, proto košík přežije přechod mezi stránkami.
3. **Obrazovky** (`src/views/`) – vždy jedna, mění se podle adresy.
4. **Sdílené komponenty** – `Field`, `SmartLink`, `Img`, `SelectMenu`, `ConfirmDialog` a další.

`SelectMenu` a `ConfirmDialog` nahrazují `<select>` a `window.confirm()`, které nejdou nastylovat. Zachovávají ovládání klávesnicí, Escape, návrat fokusu i role pro čtečky obrazovky.

3. Stav a data

Stav	Kde je	Kam se ukládá	Přežije obnovení stránky
košík, profil, objednávky, nastavení	`src/lib/store.js`	`localStorage` (`courtsy.*`)	ano
otevřený / zavřený košík	`src/context/StoreContext.jsx`	paměť (`useState`)	ne
krok a hodnoty formuláře pokladny	`src/views/Checkout.jsx`	paměť (`useState`)	ne
filtry, řazení, hledání	jednotlivé obrazovky	paměť (`useState`)	ne
zápisy na prohlídku	`src/views/PrivateTour.jsx`	`localStorage` (`courtsy.tours`)	ano
zhlédnutá úvodní animace	`src/components/layout/Intro.jsx`	`sessionStorage`	jen do zavření karty
obsah webu	`src/data/`	soubory v projektu	(neměnný)

Komponenty čtou uložený stav přes jediný hook `useStore()`. Ten je napojený na `store.js` pomocí `useSyncExternalStore`, takže stav je k dispozici hned při prvním vykreslení a všechny komponenty vidí stejná data.

Proč ne Redux: stav tvoří jeden košík, jeden profil a tři přepínače bez asynchronních operací. Knihovna by přidala kód bez přínosu a díky hooku `useStore()` jde implementace kdykoli vyměnit.

Práce s úložištěm je ošetřená: chyby (anonymní režim, plná kvóta) zachytí `try/catch`, načtená data se znovu validují a klíče jsou verzované. Smazání má dvě úrovně: `forgetMe()` (data bez nastavení) a `deleteAccount()` (vše). Obě se nejdřív ptají přes `ConfirmDialog`.

---

4. Validace a zabezpečení

Pravidla jsou v `src/lib/validation.js` jako čisté funkce (hodnota → chyba, nebo `null`): e-mail, jméno, heslo, telefon, adresa, PSČ podle země a platební karta (Luhnův algoritmus, platnost, délka CVC). Pokladna kontroluje každý krok zvlášť, chybné pole označí a přesune na něj fokus.

- heslo se po kontrole zahodí, z karty se uloží jen poslední čtyři číslice,
- text se před uložením zkrátí a zbaví řídicích znaků,
- aplikace nepoužívá `dangerouslySetInnerHTML`, React text escapuje sám.

---

5. Testování

Vrstva	Nástroj	Počet	Co ověřuje
Unit	Vitest	87	validace, store a součty, formátování dat
Komponenty	Vitest + React Testing Library	15	formulářové pole, rozbalovací nabídka, pokladna
E2E	Playwright (Chromium)	43	navigace, nákup, mazání účtu, SEO, stránka 404

E2E testy běží proti produkčnímu buildu. Nejdelší projde celý nákup až po nalezení objednávky v účtu. Jeden test hlídá, že se při procházení všech stránek nikdy znovu nenačte celý dokument, tedy že aplikace zůstává SPA.

---

6. Profilování a optimalizace

Měřeno nástrojem Lighthouse (`npm run audit`) nad produkčním buildem v emulaci mobilu. Reporty jsou v `docs/lighthouse/`.

Stránka	Výkon	Přístupnost	LCP	CLS
`/`	70 → **88**	90 → **100**	9,8 s → **3,5 s**	0 → **0**
`/memberships`	68 → **92**	94 → **100**	3,2 s → **3,1 s**	0,58 → **0**
`/courts/centre-court`	57 → **89**	96 → **100**	5,0 s → **3,3 s**	0,59 → **0**

- **Posun rozvržení (CLS):** obrazovky se načítají přes `lazy` na úrovni routy, stránka se vykreslí najednou a nic neposkočí.
- **Načtení obrázků (LCP):** fotografie se převádějí do WebP ve čtyřech šířkách, hlavní obrázek klesl z 293 kB na 72 kB.
- **Písmo** je součástí buildu místo načítání z Google Fonts.
- **Přístupnost:** opravený kontrast, pořadí nadpisů, velikost ovládacích prvků a omezení pohybu i pro CSS animace.

---

7. SEO

SPA posílá pro všechny adresy stejný `index.html`. Proto každá obrazovka volá hook `useSeo()` (`src/lib/seo.js`), který nastaví vlastní titulek, popis, kanonickou adresu a náhled pro sociální sítě. Pokladna, účet a 404 mají `noindex`, mapa webu `sitemap.xml` se generuje při buildu.

Výsledek ověřuje Lighthouse (kategorie SEO 100 na všech měřených stránkách) a čtyři testy v `tests/e2e/seo.spec.js`.